

KTransformers-FineTune: Beyond Offload with Layout-Aware and Schedule-Optimized Heterogeneous MoE Fine-Tuning

Peilin Li^{1,2}, Xingxing Hao^{2,3}, Hongtao Chen^{1,2}, Weiyu Xie^{1,2}, Yaowei Zheng⁴, Bowen Wu^{2,5}, Yujie Yang^{2,6}, Huanming Shen^{1,2}, Qingliang Ou^{1,2}, Boxin Zhang^{1,2}, Jingqi Tang^{1,2}, Ziwei Yuan^{1,2}, Jianwei Dong^{1,2}, Dongdong Kuang⁴, Zhangchi Feng⁴, Jiaheng Dai², Qianrui Yang¹, Shaoyuan Chen⁷, Jiahao Wang^{2,8}, Yaochen Han^{2,4}, Yuening Zhu^{1,2}, Jiaqi Liao^{2,9}, Xianglin Chen², Zhiyuan Ai², Yongwei Wu¹, Mingxing Zhang^{1,*}

¹Tsinghua University ²Approaching.AI ³Zhejiang University ⁴Beihang University ⁵Northwestern Polytechnical University ⁶Harbin Institute of Technology, Shenzhen ⁷DeepSeek-AI ⁸Hangzhou Dianzi University ⁹Beijing Institute of Technology

Abstract

Fine-tuning trillion-parameter Mixture-of-Experts (MoE) models locally is essential for privacy-sensitive adaptation, but existing GPU-CPU offloading systems are bottlenecked by PCIe bandwidth in sparse MoE fine-tuning.

We present KTransformers-FineTune (KT-FT), a heterogeneous fine-tuning system that replaces offloading with co-computation paradigm, turning the host CPU into an active participant in MoE execution. KT-FT combines a layout-aware memory engine with skew-tolerant scheduling to eliminate repacking overheads, sustain efficient CPU execution, and avoid out-of-memory failures at long context lengths. To further improve Time-to-Quality, KT-FT introduces LoRA Experts as a orthogonal algorithm co-design, which use residual GPU memory to accelerate convergence and improve final model quality. Across trillion-parameter MoE models, KT-FT delivers 2.5×–21.2× higher fine-tuning throughput than prior offloading systems and scales to totally 128K-token contexts without OOMs. It also enables the fine-tuning of the 671B-parameter DeepSeek model (2K context) in BF16 precision using merely 16GB of GPU VRAM alongside the host CPU. As a drop-in backend for existing fine-tuning ecosystems, KT-FT has already adopted in open-source communities, small and medium-sized enterprises, and foundation-model teams, making privacy-preserving adaptation of trillion-parameter MoE models practical and cost-effective.

Keywords: mixture-of-experts, GPU/CPU hybrid fine-tuning, large language models, LoRA fine-tuning, system-algorithm co-design

1 Introduction

Customizing foundation models on proprietary or sensitive data [18, 53], including healthcare records [12, 26, 52] and financial ledgers [19, 40, 59], is increasingly essential for academic research and enterprise applications. However, data privacy, security, and strategic requirements often dictate

*Corresponding author: Mingxing Zhang.
Email: zhang_mingxing@mail.tsinghua.edu.cn.

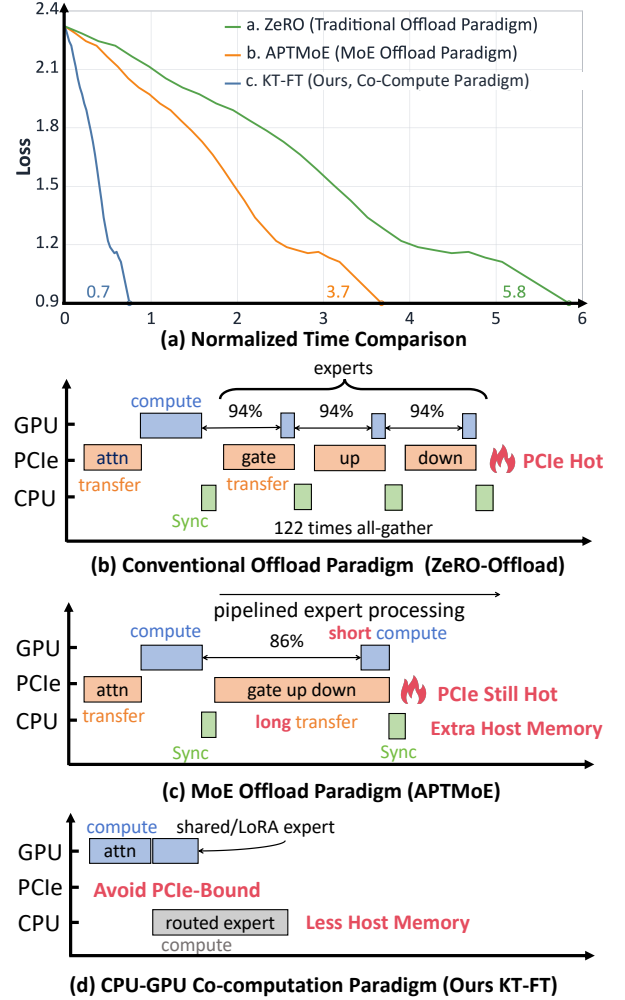


Figure 1. Conventional offload paradigm versus KT-FT’s co-computation paradigm. Attention is abbreviated as attn.

that these workloads execute strictly on local, on-premises infrastructure [18]. Systems running these workloads therefore face a resource bottleneck from the large memory demands of modern large language models (LLMs). A typical

full-parameter fine-tuning run must hold model weights, gradients, and optimizer states [34, 35], increasing its memory footprint by up to $9\times$ relative to inference [76]. Consequently, fine-tuning today’s frontier trillion-parameter models (e.g., Kimi K3 [22], GLM-5.2 [13], and DeepSeek-V4 [8]) requires tens of terabytes (TB) of GPU memory.

Even with parameter-efficient fine-tuning (PEFT) methods such as LoRA [17], adapting trillion-parameter foundation models needs substantial GPU memory, typically 2–3 TB in aggregate. An on-premises deployment with this much GPU memory requires more than thirty H100 GPUs and costs millions of dollars. For private clusters, GPUs with high-capacity HBM remain expensive and supply-constrained, and access to them is unavailable through short-term leases. Consequently, model customization remains out of reach for most research labs [1] and medium-sized enterprises [50]. Memory-efficient systems are therefore urgently needed.

To work around strict GPU memory limits, existing systems such as ZeRO-Offload [46] and StrongHold [51] use CPU–GPU offloading. As Figure 1b shows, this offload paradigm stores parameters and optimizer states in host memory and pages working sets into the GPU over PCIe. Its effectiveness depends on overlapping communication with computation. For sparse Mixture-of-Experts (MoE) models, this assumption breaks down because expert weights dominate the working set while sparse routing reduces arithmetic intensity. Our profiling of DeepSeek-V3.1-671B [7] on an RTX 4090 shows that ZeRO-Offload transfers roughly 2.6 TB of weights and performs 122 all-gathers per step. Consequently, PCIe communication and dispatch account for 94% of step time, leaving GPU utilization at only 0.8%.

Recent systems such as ES-MoE [21], MoESys [63], and APTMoE [57] schedule computation and weight transfers at expert granularity to improve computation–communication overlap, including streaming expert weights, hotness-aware prefetching, and pipelined dispatch. However, our profiling reveals that these scheduling techniques in APTMoE add additional scheduling overhead for 11% of execution time. Moreover, these techniques increase host memory required for Qwen3-235B [61] from 500 GB to 1.3 TB and cause DeepSeek-671B to run out of memory (OOM) on a 2 TB server. Despite the improved overlap, Figure 1c shows that weight transfer and synchronization still account for 86% of APTMoE’s execution time and remain the dominant execution costs. Even if the conventional offload paradigm were to utilize an idealized PCIe 5.0 interconnect at theoretical peak bandwidth, required weight movement would limit theoretical throughput to roughly 48 tokens/s (2K context) and GPU utilization to about 3.9%, as quantified in §2.1. Scaling MoE fine-tuning to trillion-parameter models therefore requires abandoning the conventional offload paradigm.

Given this PCIe bottleneck, we observe computing expert layers on CPU with AVX/AMX extensions is faster than

transferring their weights over PCIe. This observation necessitates a paradigm shift toward CPU–GPU co-computation to exploit data locality. However, simply moving expert computation to CPU is not sufficient. A naive co-computation implementation in PyTorch(OneDNN) has lower end-to-end throughput than offloading paradigm over long sequences. Our profiling identifies layout overheads from incompatible memory layouts (§2.2) and synchronization overheads from CPU thread imbalance under skewed expert routing (§2.3). We establish the necessary condition that CPU computation time plus these overheads must be less than expert weight transfer time.

To overcome these bottlenecks, we present KT-FT, a transparent execution engine that enables trillion-parameter MoE LoRA fine-tuning on memory-constrained GPUs. KT-FT runs compute-intensive attention on the GPU and memory-bound MoE experts on the CPU to avoid repeated expert-weight transfers. Moreover, KT-FT makes this co-computation paradigm practical via two components that reduce the identified layout and synchronization overheads:

- **Semantic-Aware Proactive Layout Engine:** The PyTorch(OneDNN) implementation relies on a fixed tensor layout and therefore performs layout transformations on the critical path. KT-FT selects a layout for each tensor based on its semantics, as determined by its execution device, fine-tuning stage, and parameter size. To achieve this, KT-FT introduces two techniques: a semantic-aware memory pool that allocates tensors in the layouts required by each phase, and an asynchronous pipeline across model layers that hides the latency of layout transformations by overlapping them with GPU computation during the forward and backward passes. Together, these layout optimizations deliver a $7.9\times$ speedup over PyTorch(OneDNN) baselines without increasing the memory footprint. For optimizer updates, KT-FT proposes a memory layout that packs LoRA parameters and optimizer states across experts into large contiguous buffers, enabling sequential access and batched updates using vectorized operations. This layout delivers a $13.9\times$ speedup for parallel optimizer updates.
- **Skew-Tolerant Dynamic Thread Scheduler:** To address CPU thread imbalance under MoE routing skew, KT-FT’s dynamic scheduler decomposes the workload into fine-grained tasks that can be scheduled across expert boundaries. Workers pull these tasks from a shared queue using dynamic work stealing, distributing computation for hot experts across multiple workers while tasks from the long tail of cold experts fill otherwise idle CPU cycles. The task decomposition is also cache-aware. To improve temporal locality, KT-FT splits memory-bound base-weight computation along weight dimension to avoid L3 thrashing and

splits LoRA computation along sequence dimension to reuse weights that fit in L2. By reducing dispatch overhead and synchronization barriers, the scheduler achieves a 7 $\times$ speedup for the backward-pass operator over the PyTorch(OneDNN) baseline.

For standard LoRA, these two components enable KT-FT to achieve 2.5 $\times$ –21.2 $\times$ the throughput of offloading baselines and support 128K contexts without out-of-memory errors. As an orthogonal optimization, we seek to further minimize Time-to-Quality (TTQ) [4, 5, 36] using residual GPU memory. After the GPU working set is allocated, a small fraction of VRAM remains unallocated, typically less than 5 GB. However, leveraging this scarce memory through purely system-level mechanisms yields limited benefits. Dynamic expert migration is bottlenecked by PCIe and layout overheads, whereas static expert placement accommodates only a small subset of routed experts. Across all evaluated settings, TTQ reductions from both dynamic expert migration and static placement remain below 10%.

Instead of spending this limited budget on these system-only mechanisms, KT-FT allocates it to LoRA Expert (LE), an orthogonal and optional algorithm–system co-design. LE uses residual GPU memory to keep its weights resident on the GPU and provides a static shared adaptation path executed for every token, regardless of expert routing decisions. LE avoids routed expert migration, and its GPU computation fully overlaps with computation by routed experts on the CPU. As a result, LE does not increase end-to-end time. From an algorithmic perspective, LE uses this dense shared gradient and update path to mitigate the update imbalance induced by routing skew and improve overall data utilization. Reallocating 1.2 GB of residual VRAM to LE reduces TTQ by a factor of at least 1.9 relative to both the Routed Control and the Shared Control. At full convergence, KT-FT outperforms the base models on all reported quality measures and both controls on most metrics across the evaluated models and workloads. LE is designed to work with a wide variety of LoRA methods, and we evaluate it using four representative variants: Standard LoRA [17], PLoP [15], AltLoRA [65], and Bi-LoRA [32].

We implement KT-FT as a transparent execution engine integrated with HuggingFace Accelerate [14] and LlamaFactory [76] without requiring users to rewrite code. It supports LoRA fine-tuning, full-parameter fine-tuning, and reinforcement learning workflows and has been adopted by open-source communities. We deploy KT-FT for a small or medium-sized enterprise on one production node with limited GPU resources. Compared with a baseline framework on 16 A100 GPUs, KT-FT on 8 RTX 4090 GPUs is 1.6 $\times$ faster and reduces per-iteration compute cost by 81.5%.

2 Motivation

2.1 The Limits of Offloading in Sparse MoE Systems

Conventional offloading systems such as ZeRO-Offload [46] place the model state in CPU memory, reducing GPU memory use at the cost of CPU–GPU transfers over PCIe [45, 46, 51]. These systems are efficient only when GPU computation is sufficient to hide CPU–GPU transfer latency and synchronization overhead. In MoE models, however, sparse routing activates about 5% of the model parameters for each token. This reduces the ratio of GPU computation to data transferred by roughly 20 $\times$ and leaves 88% of communication time on the critical path. Consequently, the conventional offload paradigm cannot hide most of the overhead from CPU–GPU transfers and synchronization during MoE fine-tuning.

To address this limitation, prior offloading systems designed for MoE models use techniques including streaming expert weights to reduce synchronization overhead [21], hotness-aware prefetching to move weights that are likely to be needed onto GPUs before execution [63], and pipelined dispatch to overlap expert loading with computation [57]. However, our profiling of APTMoE [57] shows that scheduling overhead accounts for 11% of its execution time. These scheduling mechanisms also increase the host memory required for Qwen3-235B [61] from 500 GB to 1.3 TB and cause DeepSeek-671B OOM error on a 2 TB server. Despite the improved overlap, weight transfer and synchronization account for 86% of APTMoE’s execution time and remain the dominant execution costs.

Calculation shows that even with an ideal scheduler and perfect overlap, the amount of GPU computation is too low to hide the latency of the required weight transfers. For 1 billion BF16 parameters, the total amount of data transferred during the forward and backward passes is 4 GB. At the theoretical peak bandwidth of PCIe 5.0 $\times 16$ (64 GB/s), this transfer takes 62.5 ms. The total computation time for the corresponding forward and backward passes is only 2.8 ms on the RTX 5090, making the transfer more than 22 $\times$ as long as the GPU computation. More generally, the offload paradigm must transfer at least two full copies of the model weights ($2|W|$) during each training step. For DeepSeek-671B, $2|W| \approx 2,684$ GB in BF16. This transfer takes 42 seconds at the same theoretical peak bandwidth. At 2K tokens per step, the transfer bottleneck limits throughput to roughly 48 tokens/s and the theoretical utilization of the RTX 5090 to 3.9%. KT-FT reaches 166.8 tokens/s, 3.5 $\times$ the theoretical throughput limit of the offload paradigm. Thus, our co-computation paradigm needs to avoid transfers of expert weights.

KT-FT therefore executes expert layers on the CPU, avoiding repeated transfers of expert weights to the GPU. For each MoE layer in DeepSeek-671B, transferring the required expert weights takes roughly 22 ms (T_{transfer}), whereas CPU expert computation takes 5.8 ms ($T_{\text{CPU compute}}$). However,

at long sequence lengths, our CPU-GPU co-computation paradigm alone is insufficient to improve end-to-end performance. In our ablation at 2K tokens, a PyTorch(OneDNN) implementation of our paradigm reaches 12.1 tokens/s, below ZeRO-Offload’s 41.6 tokens/s. Accordingly, a necessary condition for CPU-GPU co-computation to improve end-to-end performance is $T_{\text{CPU compute}} + T_{\text{layout}} + T_{\text{sync}} < T_{\text{transfer}}$. In other words, layout and synchronization overhead must together remain below the time saved by replacing the transfer of expert weights with local CPU computation. Our profiling shows that T_{layout} stems from incompatible layouts required by different tensor semantics, forcing computation with poor cache locality or layout conversions during fine-tuning (§2.2). T_{sync} stems from scheduling and synchronization overheads caused by CPU workload imbalance under routing skew (§2.3).

2.2 Memory Layout Conflicts

The CPU-GPU co-computation paradigm mitigates the PCIe bottlenecks by computing expert layers on the CPU. Inference engines [2, 20] successfully exploit CPU extensions such as AVX/AMX for this purpose. Extending the same paradigm to fine-tuning, however, introduces serious system bottlenecks. Unlike forward-only inference, fine-tuning requires both forward and backward passes through a multi-stage computation graph. The graph contains tensors with different lifetimes and precisions, including low-precision frozen base weights, high-precision trainable LoRA parameters, and mixed-precision optimizer states. These tensors must persist across execution stages, but the memory access pattern best suited to each tensor changes with the execution stage and target hardware. These changes create memory layout conflicts that contribute to T_{layout} . Resolving these conflicts globally is therefore necessary to sustain high CPU utilization.

Cross-Device Layout Conflicts. Hardware heterogeneity creates a layout conflict at the device boundary, where GPU and CPU kernels require different tensor layouts. GPU Tensor Cores operate efficiently on row-major arrays. Efficient AVX/AMX execution on the CPU requires VNNI blocks that keep the reduction dimension contiguous and store BF16 values in pairs. The PyTorch implementation [43] stores all weight matrices in row-major format, so the output dimension is contiguous in memory but the reduction dimension is not. Consequently, every GEMM invocation uses strided loads to access non-contiguous memory, reducing cache utilization and the fraction of valid elements in each tile. Figure 2a shows the effect on end-to-end MoE backward-pass throughput: PyTorch reaches at most 1.8 TFLOPS and falls below 50 GFLOPS at small per-expert batch sizes. KT-FT converts weights to VNNI block layouts at load time and keeps these layouts throughout fine-tuning. This removes the per-call cost of strided memory access. It reaches 15.7 TFLOPS, up to 11.7× the throughput of the PyTorch baseline.

Cross-Stage Layout Conflicts. Forward and backward computation have incompatible layout requirements even on the same device. The forward pass uses VNNI blocks, but the transposed GEMMs in the backward pass cannot use the same layout directly. VNNI blocks store BF16 values in pairs along the reduction dimension. A straightforward approach is to change only the strides, but this does not produce the layout required by the transposed GEMMs. The weights must therefore be unpacked, transposed, and repacked. Existing engines must either retain an additional representation for transposed access, at the cost of increased memory use, or use a single layout that is suboptimal for both directions. The PyTorch implementation chooses to use a single row-major weight layout for both forward and backward computation. In Figure 2b, the orange curve falls to 57 GFLOPS at small batch sizes and peaks at 9.7 TFLOPS. At 1K tokens per expert, PyTorch achieves 52% lower throughput than KT-FT. KT-FT instead uses the packed layout required by each access direction. Forward and backward GEMMs therefore use only their respective VNNI block layouts, allowing KT-FT to sustain high hardware utilization and reach up to 17.6 TFLOPS in either direction (Figure 2b, blue curve).

LoRA Parameter Layout Conflicts. Applying LoRA to MoE models creates parameter fragmentation. Each expert has separate LoRA matrices for its gate, up, and down projections. The forward and backward passes use these matrices for per-expert GEMMs, while optimizer updates benefit from fused contiguous storage across experts. These requirements create a layout conflict between matrix computation and parameter updates. For DeepSeek-671B with 256 experts, Figure 2c shows that parameter block count increases by 256× relative to a fused layout. Updating fragmented blocks separately makes kernel launch overhead the dominant update cost and reduces effective memory bandwidth. Figure 2d shows that optimizer step latency increases to 899 ms. Using this fused layout reduces it to 65 ms, a 13.9× improvement.

2.3 Workload Imbalance and Synchronization Overhead

Existing CPU execution stacks schedule threads within individual expert operations. The PyTorch implementation with OneDNN, for example, parallelizes each expert GEMM across CPU threads but does not schedule work across experts. Sparse MoE routing creates a skewed workload across experts, and many experts receive only a few tokens [55, 56]. Figure 3 shows this skew in layers 0 and 50 of Qwen3.5-397B, configured with 512 routed experts, top-10 routing, and a sequence length of 4K. The most heavily loaded expert receives up to 3,900 tokens, more than 40× the mean, while each expert in the long tail receives a negligible number of tokens. For lightly loaded experts, the corresponding GEMMs have a small token dimension and limited intra-operation

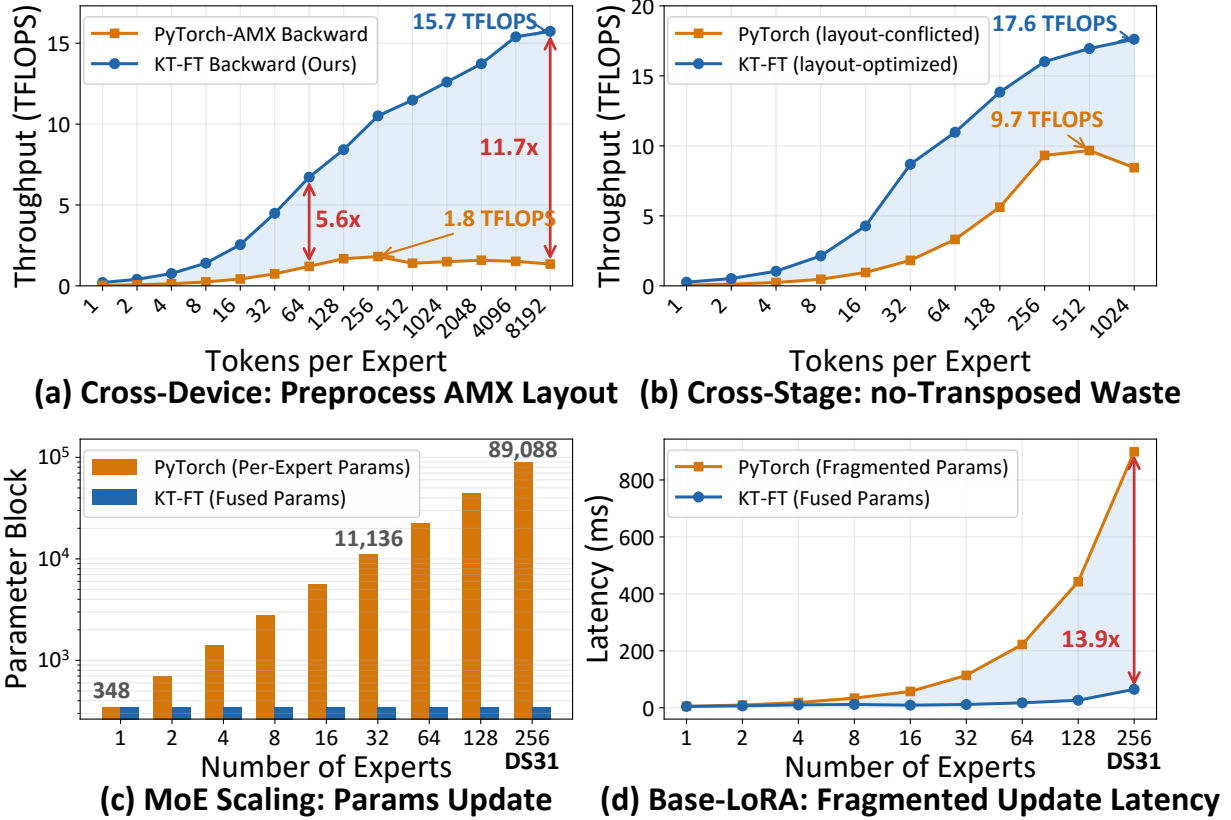


Figure 2. Memory layout conflicts and their effects on MoE fine-tuning. Panel (a) reports end-to-end MoE backward-pass throughput under the cross-device layout requirement, while panel (b) reports expert GEMM throughput under the cross-stage layout requirement; panels (c) and (d) report parameter block counts and optimizer step latency for fragmented and fused LoRA parameter layouts. All measurements use a DeepSeek-V3.1 (DS31) configuration representative of trillion-parameter MoE models.

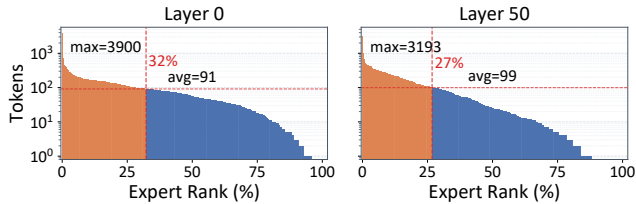


Figure 3. Token distributions across routed experts in Qwen3.5-397B with sequence length of 4K. Layers 0 and 50 are representative of the routing skew observed across all layers.

parallelism. Because scheduling remains confined to individual operations, these GEMMs cannot fully utilize AVX/AMX, and CPU threads remain unevenly loaded.

Fine-tuning introduces a separate source of overhead. It creates a separate autograd operation for each expert and each LoRA projection. Many expert operations process only a few tokens, and LoRA projections have small parameter

dimensions. These operations therefore perform little computation, but each still incurs overhead from metadata management, traversal of autograd graph, and synchronization during backward pass. Across many operations, these costs increase T_{sync} on the fine-tuning critical path. Therefore, parallelizing individual GEMMs is insufficient, the runtime must schedule fine-grained work across experts.

3 KT-FT Design

3.1 System Overview

KT-FT provides efficient, end-to-end MoE fine-tuning on heterogeneous CPU-GPU platforms through a transparent execution engine. It integrates with HuggingFace Accelerate and LlamaFactory for fine-tuning and reinforcement learning workflows. KT-FT supports full-parameter fine-tuning and all PEFT methods available in the HuggingFace ecosystem, including LoRA and its variants. The backend supports Intel and AMD CPUs with AVX2, AVX-512, or AMX extensions. As shown in Figure 4, users add only two lines to their code in

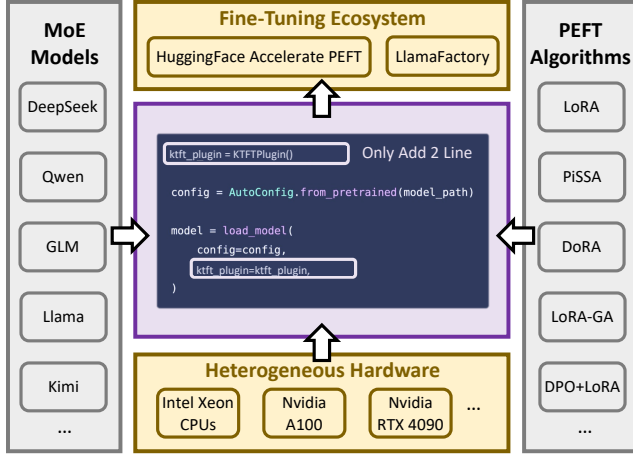


Figure 4. KT-FT’s integration with the fine-tuning ecosystem.

the `load_model`, enabling KT-FT to take over execution. KT-FT hides the hardware complexity, users launch fine-tuning tasks with a bash command just like before.

Behind this interface, KT-FT uses Fully Sharded Data Parallel (FSDP) [75] to run attention computations on the GPU. It executes routed experts on the CPU to avoid transferring expert weights, using its layout engine and thread scheduler to reduce layout and synchronization overheads.

The Semantic-Aware Proactive Layout Engine selects tensor formats at allocation time according to each tensor’s role in fine-tuning (§3.2). Its proactive pipeline switches between forward and backward weight layouts before use, overlapping conversion with GPU computation while keeping a single copy of the parameters. The engine also packs LoRA parameters and optimizer states across experts into large contiguous buffers for batched updates. Tensor views then expose each expert’s parameters for forward and backward computation without copying data.

The layout engine improves data access, but it does not determine how work is distributed among CPU threads. To handle routing skew, the Skew-Tolerant Dynamic Thread Scheduler divides each expert’s forward and backward computation into fine-grained tasks that access tensors in these layouts (§3.3). It partitions base-weight computation along weight dimensions to avoid repeated weight reads, and LoRA computation along the sequence dimension to reuse cached weights. A shared queue and dynamic work stealing distribute these tasks among CPU workers.

3.2 Semantic-Aware Proactive Layout Engine

The semantic-aware memory pool selects a layout for each tensor based on its role and execution stage; the proactive pipeline converts the parameters to the required layout before use while keeping a single parameter copy. For LoRA,

the contiguous layout supports both sequential access during optimizer updates and per-expert access during forward and backward computation.

Semantic-Aware Memory Pool. Parameter storage formats do not always match the layouts optimal for computational efficiency and cache locality. Such mismatches include cross-device conflicts, where CPU MoE kernels perform best with a layout different from the stored weight format, and cross-stage conflicts, where a layout suited to forward computation has poor cache locality in the backward pass. Figure 5 ① shows the cross-device case: PyTorch(OneDNN) stores tensors in row-major format (blue), whereas CPU matrix coprocessors require a different layout. Repacking these tensors during execution is memory-bound; it repeatedly stalls computation and prevents the CPU from reaching its theoretical throughput.

KT-FT uses a semantic-aware memory pool to resolve this mismatch. Tensor formats are selected at allocation time. The pool groups tensors by their role in the computational graph rather than by allocation lifetime: frozen weights for the forward and backward passes, trainable LoRA parameters and their optimizer states for individual experts, and temporary activation buffers.

Each memory pool specifies an allocation policy and the data format best suited to the target hardware and execution stage. For the CPU forward pass, the Layer 0 pool in Figure 5 ④ allocates weights in VNNI blocks (green), so AMX instructions can use them directly. For the backward pass, the same weights are stored in transposed VNNI blocks (peach) to optimize cache locality during gradient computation. Selecting layouts according to tensor semantics at allocation time removes repeated format conversions from the execution path.

Proactive Layout Pipeline. The memory pool selects the optimal layout for each phase, but switching between the forward and backward passes still requires layout conversion (Figure 5 ②). VNNI data must be unpacked, transposed, and repacked, so performing this conversion during execution adds latency. Precomputing and storing both layouts avoids this cost but doubles the memory used for parameters, requiring 1–2 TB of host memory for models at the trillion-parameter scale, such as DeepSeek-671B.

KT-FT uses a proactive layout pipeline to switch between the two layouts while keeping a single copy of the parameters. The execution graph follows a deterministic layer order, allowing background threads to transform each layer’s layout in place before it is needed. These transformations overlap with GPU computation (Figure 5 ⑤) and stay off the critical path.

Contiguous Layout for LoRA Optimizer Updates. At trillion-parameter scale, LoRA parameters from tens of thousands of experts occupy more than 100,000 separate blocks. The compute-bound forward and backward passes hide the latency of scattered accesses, whereas the memory-bound,

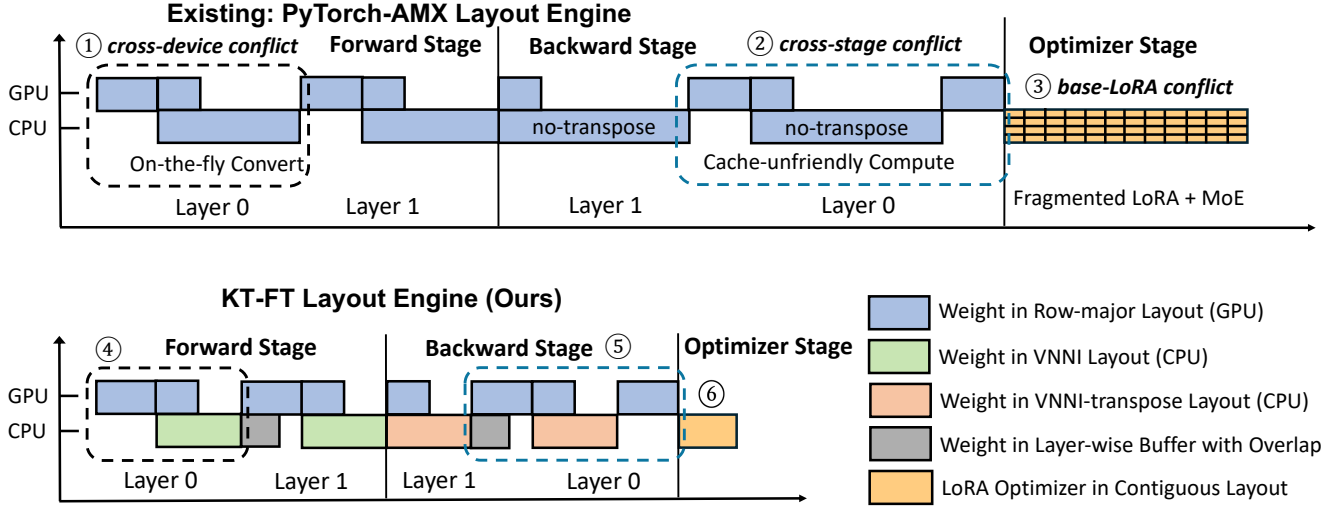


Figure 5. System overview of the Semantic-Aware Proactive Layout Engine. The forward and backward passes use different layouts for the same parameter values in each layer (e.g., Layer 0).

write-intensive optimizer step remains sensitive to data layout (Figure 5 ③). Poor spatial locality and pointer chasing limit optimizer performance.

KT-FT uses an contiguous layout to address this memory bandwidth bottleneck. Figure 5 ⑥ shows how KT-FT packs LoRA parameters and optimizer states across experts into large contiguous buffers (orange) for sequential access and batched updates using vectorized operations. During the forward and backward passes, the semantic-aware memory pool provides contiguous tensor views of each expert’s parameters within these buffers. The views require no data copies and preserve the data structures and formats used for forward and backward computation, while the optimizer accesses contiguous memory.

3.3 Skew-Tolerant Dynamic Thread Scheduler

Under routing skew, KT-FT uses fine-grained scheduling across experts to balance work across CPU worker threads (§2.3). KT-FT replaces sequential expert execution (the upper panel of Figure 6) with a flat pool of fine-grained tasks (the lower panels). The scheduler selects individual tasks across expert boundaries. Table 1 shows how forward and backward computation is divided into tasks along the token (Q), active expert ($|\mathcal{A}|$), and tile (P) dimensions to expose parallelism. This common representation supports separate scheduling policies for base-weight and LoRA computation.

The two decompositions in Figure 6 ① and ② reflect the different memory behavior of base-weight and LoRA computation:

- **Base-Weight Computation (Splitting Along Weight Dimensions):** Base-weight computation is limited by memory bandwidth. Splitting this work

along the sequence dimension would fetch the same weights repeatedly and cause L3 cache thrashing. KT-FT therefore splits the work along weight dimensions (e.g., hidden channels), so each thread streams its assigned weight tile once over the full sequence.

- **LoRA Computation (Splitting Along the Sequence Dimension):** The compact LoRA adapters fit entirely in L2 cache. KT-FT splits LoRA computation along the sequence dimension. Each thread streams a partition of the token activations while reusing the weights in L2 cache, maximizing temporal locality and arithmetic intensity.

These cache-aware tasks can still leave workers unevenly loaded under routing skew. KT-FT therefore places the tasks in a shared queue and uses dynamic work stealing to distribute them among workers. Multiple workers process tiles from hot experts, while work from lightly loaded experts fills otherwise idle CPU cycles. By reducing dispatch stalls and avoiding the overhead of autograd metadata, KT-FT achieves a $7\times$ speedup over PyTorch(OneDNN) for the backward-pass operator.

4 LoRA Expert

4.1 Motivation: Residual GPU Memory

The system optimizations in the previous section improve throughput for standard LoRA fine-tuning. After allocating the dense working sets, KT-FT typically leaves a small fraction of GPU memory unused (Figure 7a). This residual GPU memory can be used to further accelerate training and reduce TTQ [4, 5, 36].

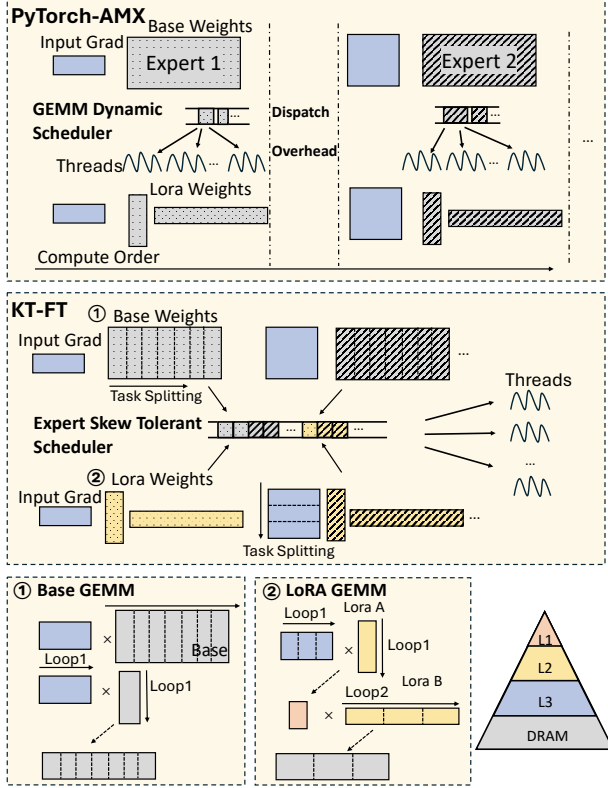


Figure 6. The upper panel shows PyTorch(OneDNN) executing experts sequentially and parallelizing each GEMM across CPU threads. The lower panels show KT-FT’s Skew-Tolerant Dynamic Thread Scheduler dividing base-weight and LoRA computation into a shared pool of tasks according to their memory behavior.

Prior system-level approaches use the remaining GPU memory for dynamic expert migration or static expert placement. Both strategies rely on overlapping data movement with computation and on predictable expert routing. Even with accurate predictions, PCIe transfers and layout conversion for dynamic expert migration take longer than executing the experts on the CPU. Static placement avoids transfer overhead but requires a stable, predictable set of hot experts that fits in the memory budget. Routing demand is difficult to predict, and this budget accommodates only a small subset of routed experts. The best placement we evaluated reduces TTQ by less than 10%.

KT-FT instead takes an algorithm–system co-design approach. Under domain shift, downstream data induces concentrated, task-dependent routing [55]. Updates during fine-tuning are therefore concentrated on a small set of hot experts, while many cold experts receive few updates [56]. These observations motivate using the limited GPU memory for the shared, routing-independent adaptation path of LoRA Expert (LE) [6, 33], rather than marginally increasing

Table 1. Representative task decomposition for the forward and backward passes. $|\mathcal{A}|$ is the number of active experts, and P is the number of tiles per expert determined by the AMX kernel. Q is the token count, and N_t is the LoRA parallelism along the sequence dimension. B_t is the token block size, and Q_i is the token count of the i -th active expert.

Direction	Phase	Dimension	Task Count
Forward	Input gathering	token	Q
	Input quantization	expert	$ \mathcal{A} $
	Gate+Up GEMM	expert×tile	$ \mathcal{A} \times P \times 2$
	Gate+Up LoRA	expert×token	$ \mathcal{A} \times N_t \times 2$
	Down GEMM	expert×tile	$ \mathcal{A} \times P$
	Output merging	token	Q
Backward	Gradient scattering	token	Q
	Intermediate gradient	expert×tile	$ \mathcal{A} \times P$
	LoRA gradient projection	expert×token	$\sum_i [Q_i / B_t]$
	Activation backward	expert	$ \mathcal{A} $
	Input gradient	expert×tile	$ \mathcal{A} \times P \times 2$
	LoRA input gradient	expert×token	$ \mathcal{A} \times N_t \times 2$
	Gradient accumulation	token	Q

routed capacity. This path captures common post-training signals for all tokens while preserving the task-specific routing structure established during pre-training. KT-FT runs LE on the GPU concurrently with routed experts on the CPU, improving adaptation and reducing TTQ without increasing the time per training step.

4.2 Design: Static Shared Adaptation Path

LE provides an optional shared adaptation path for each MoE layer in the CPU–GPU co-computation paradigm. It runs on the GPU in parallel with the routed experts on the CPU and is executed for every token. LE requires no additional routing decision and leaves the original MoE router unchanged. Its down projection is initialized to zero. For an input hidden state h , LE computes a shared representation:

$$h_{LE} = W_{down}^{LE} \left(\text{SiLU}(W_{gate}^{LE} h) \odot W_{up}^{LE} h \right).$$

The LE output is added to the output of the experts on the CPU: $h_{out} = h_{MoE} + h_{LE}$. LE’s weights remain in GPU memory throughout training.

Where LE is attached and how it is computed do not depend on the LoRA update rule. It can be enabled independently across LoRA variants without changing their mathematical update rules.

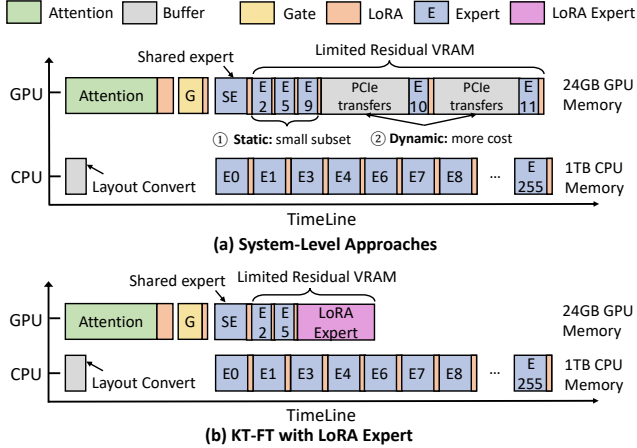


Figure 7. Allocation of residual VRAM. A small subset of hot experts is statically cached, and the remaining capacity is allocated to LE. LE is an optional, routing-independent shared adaptation path. It remains on the GPU and runs in parallel with routed experts on the CPU, reducing TTQ.

5 Evaluation

To evaluate KT-FT’s performance and analyze how its system mechanisms contribute to the improvements, we answer the following questions:

- Q1.** Can KT-FT’s co-computation paradigm improve end-to-end fine-tuning throughput and support larger context windows than existing offloading systems? (§5.2)
- Q2.** How much do the layout engine and thread scheduler each contribute to CPU computation efficiency across training stages? (§5.3)
- Q3.** Does allocating residual GPU memory to LE reduce TTQ and improve final model quality? Are the improvements consistent across different LoRA variants? (§5.4)

5.1 Experimental Setup

Hardware. We use an Intel platform with two Xeon Gold 6454S CPUs supporting AMX and an AMD platform with two EPYC 9355 CPUs supporting AVX-512 to cover different CPU instruction sets. We evaluate KT-FT across two generations of CPU–GPU interconnects, using PCIe 4.0 on Intel and PCIe 5.0 on AMD. For multi-GPU execution, the Single-Node setups use 8× L40 GPUs and 2 TB of host memory on Intel, and 8× RTX 5090 GPUs and 1.5 TB of host memory on AMD, covering different GPU classes. The Single-GPU setups each have 1 TB of host memory and use 1× RTX 4090 GPU on Intel and 1× RTX 5090 GPU on AMD.

Baselines. We compare KT-FT with two classes of baselines to evaluate its architectural and system-level design choices.

- **Offloading Systems:** We use ZeRO-Offload [46], APTMoE [57], and MegaTrain [67] to compare KT-FT’s CPU–GPU co-computation paradigm with

Table 2. Architectures and parameter distributions of the evaluated MoE models. GPU and CPU allocations show parameter placement in our experiments.

Configuration	DS-31	QW-35	QW-30
Model Architecture			
MoE Layers	58	60	94
Routed Experts	256	512	128
Routing Strategy	Top-8	Top-10	Top-8
Parameter Distribution			
Total Parameters	671B	397B	235B
LoRA Parameters ($r = 8$)	3.4B (~0.5%)	3.8B (~1%)	1.7B (~0.7%)
Active Parameters	37B (~5%)	17B (~4%)	22B (~9%)
Deployment Allocation			
GPU Parameters	17B	10B	8B
CPU Parameters	654B	387B	227B

the offloading paradigm. For each evaluated setting, we enable their key optimizations and tune their parameters, using the best-performing configurations obtained. We tune bucket sizes, execution overlap, stage-3 prefetching, and staging buffers for ZeRO-Offload, and hot-expert selection, prefetch windows, and dispatch granularity for APTMoE. MegaTrain’s Single-GPU configurations use double-buffered GPU layer templates.

- **PyTorch(OneDNN):** We use a PyTorch [43] implementation of the same CPU–GPU co-computation paradigm as KT-FT, with attention on the GPU and expert computation on the CPU. We use this baseline to isolate the gains from KT-FT’s system optimizations from those due to hardware instruction set acceleration. The implementation uses OneDNN [41] for CPU expert computation and PyTorch autograd for automatic differentiation. Like KT-FT, it uses AMX on Intel and AVX-512 on AMD.

In the LoRA comparisons, KT-FT and all baselines use gradient checkpointing.

Models. We evaluate KT-FT on three frontier MoE models: DeepSeek-V3.1-671B [7] (DS-31), Qwen3.5-397B [44] (QW-35), and Qwen3-235B-2507 [61] (QW-30), as detailed in Table 2. For the throughput comparisons with the baselines, we evaluate DS-31 and QW-35 in full precision (BF16) on the Single-Node setup and DS-31 in Int8/FP8 quantization on the Single-GPU setup. For Single-GPU setup, although KT-FT can run QW-35(BF16), all offloading baselines cause OOM error. We therefore use QW-30(BF16) for the Single-GPU throughput comparisons on both platforms.

Datasets. To evaluate LoRA fine-tuning efficiency and downstream task performance, we use CoSer [54] and BeyondDialogue [66] for personalized dialogue generation and Llmrr [60] for adaptation to specialized domains. Math and code

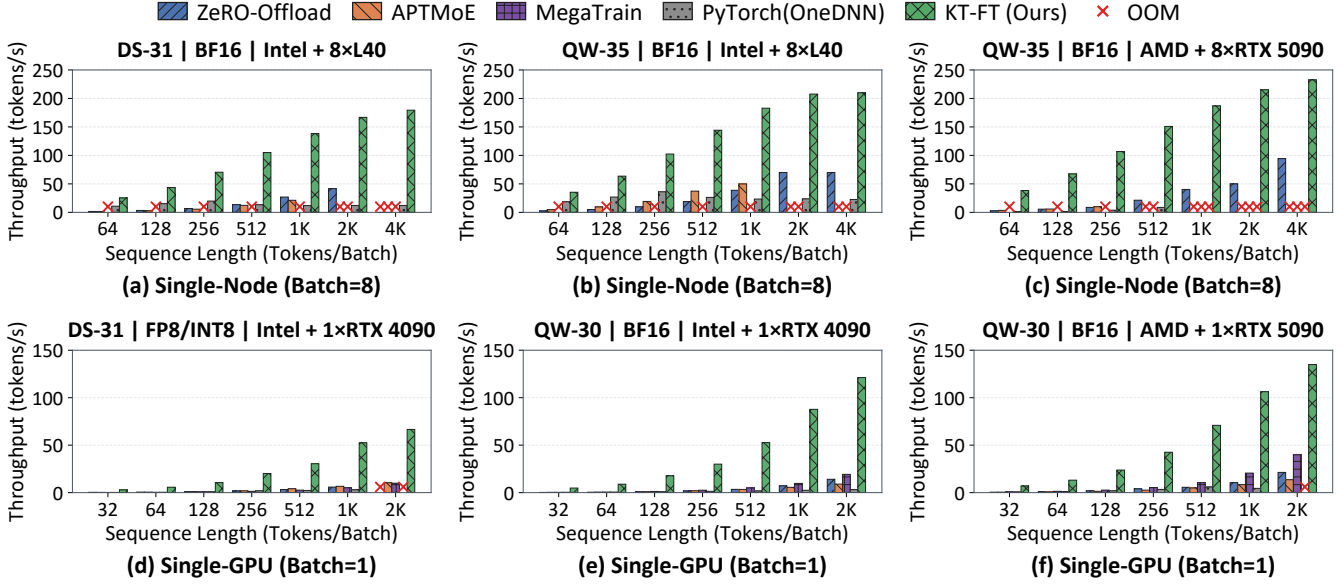


Figure 8. Fine-tuning throughput across sequence lengths.

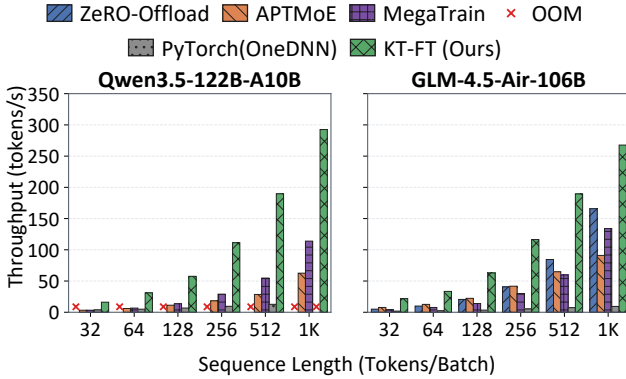


Figure 9. Full-parameter fine-tuning on Single-Node.

datasets are excluded because the frontier models have already overfit to those domains, which could obscure differences in fine-tuning efficiency.

LoRA Configurations. Following LoRA fine-tuning practices [47], we apply low-rank adaptation to all linear modules with rank $r = 8$, scaling factor $\alpha = 16$, and a dropout probability of 0.1. We use a learning rate of 10^{-4} with a cosine scheduler and a 10% warm-up ratio.

5.2 End-to-end Performance

To evaluate how KT-FT addresses the limitations of existing offloading systems, we compare end-to-end throughput (tokens/s) and memory efficiency across sequence lengths.

Throughput Comparison across Paradigms. By using CPU-GPU co-computation to avoid repeated transfers of

expert weights, KT-FT achieves $5.3\times$ – $21.2\times$ the throughput of the offloading baselines at sequence lengths below 512 tokens, as shown in Figure 8. As longer sequences require more computation per transferred weight, KT-FT’s relative throughput gain diminishes, but its end-to-end throughput remains at least $2.5\times$ that of the offloading baselines. The memory capacity of the offloading baselines determines the maximum sequence length for these throughput comparisons.

Scalability and OOM Avoidance. The bottleneck for offloading systems shifts from PCIe bandwidth to memory capacity as sequence length increases. The offloading baselines use large pipeline buffers to overlap computation and communication. These buffers increase memory consumption and cause the baselines to run out of memory at longer sequence lengths. By eliminating these large buffers and fragmented memory allocations, KT-FT completes LoRA fine-tuning with 128K-token sequences in total. Moreover, KT-FT can fine-tune DS-31 with LoRA in BF16 at a sequence length of 2K tokens using only 16 GB of GPU memory.

Full-Parameter Fine-Tuning. We focus on LoRA because it reduces memory requirement of gradients and optimizer states [17], allowing community and production users to explore trillion-parameter frontier models. Although KT-FT includes LoRA-specific optimizations, its CPU-GPU co-computation paradigm and system optimizations also improve the efficiency of full-parameter fine-tuning. We evaluate full-parameter fine-tuning of 100B-class models on the Single-Node setup because it has the largest host memory capacity among our experimental platforms. Theoretically,

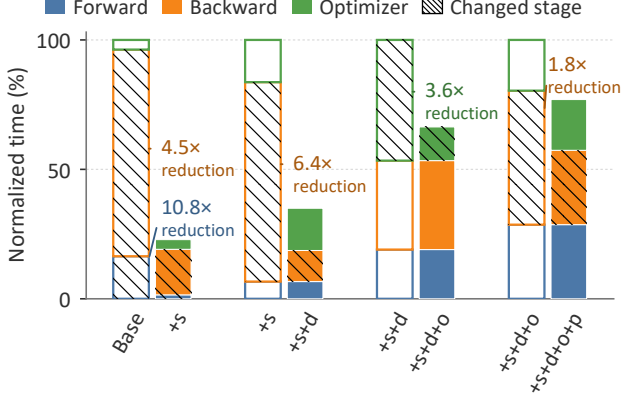


Figure 10. DS-31 LoRA ablation. Each pair shows the previous (unfilled) and current (filled) configurations, with times normalized to the previous configuration’s total step time. Numbers denote stage time reduction factors. Base is PyTorch(OneDNN), and the final configuration is KT-FT (Ours).

matrix operations require about 1.5× as many FLOPs in full-parameter fine-tuning as in LoRA. Because the two methods require similar amounts of computation, our theoretical analysis of the CPU–GPU co-computation paradigm also applies to full-parameter fine-tuning, where CPU expert computation takes less time than transferring expert weights over PCIe. Layout engine still reduces layout conversion overhead, and thread scheduler balances expert work across CPU threads under routing skew. As shown in Figure 9, KT-FT therefore retains a throughput advantage, achieving at least 1.6× the throughput of the offloading baselines.

5.3 Ablation Study of System Optimizations

Adopting the CPU–GPU co-computation paradigm alone is insufficient for efficient LoRA fine-tuning. For example, at 2K tokens length in Figure 8(a), PyTorch(OneDNN) reaches an end-to-end throughput of 12.1 tokens/s, far below than 41.6 tokens/s for offloading baseline, whereas KT-FT reaches 166.8 tokens/s after the system optimizations. To examine the contribution of each optimization, we add it successively to PyTorch(OneDNN) shown in Figure 10.

The memory pool (+s) improves spatial locality and reduces the instruction overhead of loading weights for each expert GEMM, allowing CPU kernels to use AMX/AVX more efficiently. These savings apply to both passes, yielding a 10.8× forward speedup and a 4.5× backward speedup.

Adding dynamic scheduling (+d) speeds up the backward pass by another 6.4×, with little change to the forward pass. The backward pass benefits more because it incurs additional overhead for autograd graph traversal, dependency tracking, and synchronization during gradient accumulation. For experts receiving few routed tokens and for low-rank LoRA projections, each gradient operation performs little computation, so these overheads account for a large fraction

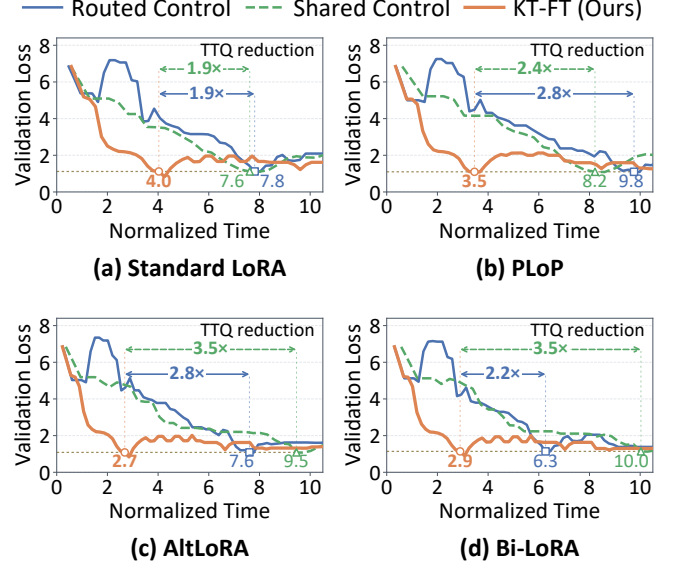


Figure 11. Loss convergence of KT-FT (Ours) and the two controls across LoRA variants.

of its execution time. KT-FT replaces per-operation autograd execution with backward tasks that it schedules across experts, reducing framework overhead and keeping CPU threads busy despite uneven expert loads.

The previous optimizations reduce expert computation time, but optimizer time remains unchanged and now accounts for 46.6% of the step time. Although LoRA updates only a small fraction of model parameters, processing many small matrices separately incurs substantial memory access and per-operation overhead. The contiguous layout (+o) enables the optimizer to access memory sequentially and amortize per-operation overhead through batched updates, speeding up the optimizer stage by 3.6×.

Furthermore, weight layout conversion remains on the critical path of the backward pass. The proactive layout pipeline (+p) fully hides this latency, speeding up the backward stage by another 1.8× for DS-31 without additional host memory. Our measurements show that CPU layout conversion takes less time than the GPU attention backward pass even for sequences as short as 16 tokens.

5.4 LoRA Expert

In this section, we answer whether allocating residual GPU memory to LE reduces TTQ and improves final model quality. To evaluate improvements across LoRA variants, we choose three broad classes of modifications to standard LoRA: target module selection (PLoP [15]), auxiliary training branches (Bi-LoRA [32]), and optimizer update rules (AltLoRA [65]).

For all LoRA algorithms, we use the same two control designs to show that simply increasing the LoRA rank in either existing adaptation path cannot achieve the same TTQ

Workloads	BeyondDialogue (Personalized Chat)							Llemr (Domain-Specific Adaptation)						
	B-1 ↑	B-2 ↑	B-3 ↑	B-4 ↑	R-1 ↑	R-L ↑	D-1 ↑	B-1 ↑	B-2 ↑	B-3 ↑	B-4 ↑	R-1 ↑	R-L ↑	D-1 ↑
QW-35 Base	0.424	0.290	0.208	0.156	0.467	0.258	0.018	0.083	0.038	0.021	0.013	0.119	0.094	0.124
QW-35 (Routed)	0.377	0.283	0.225	0.183	0.380	0.263	0.019	0.281	0.199	0.152	0.118	0.529	0.520	0.104
QW-35 (Shared)	0.393	0.305	0.247	0.205	0.462	0.315	0.023	0.384	0.280	0.219	0.175	0.552	0.538	0.164
QW-35 (KT-FT)	0.530	0.413	0.336	0.280	0.538	0.368	0.019	0.425	0.319	0.256	0.208	0.581	0.565	0.205
DS-31 Base	0.184	0.136	0.106	0.086	0.303	0.209	0.017	0.113	0.060	0.037	0.023	0.179	0.159	0.176
DS-31 (Routed)	0.493	0.388	0.318	0.268	0.531	0.370	0.019	0.345	0.247	0.189	0.147	0.582	0.572	0.160
DS-31 (Shared)	0.467	0.365	0.298	0.250	0.505	0.350	0.019	0.376	0.271	0.207	0.162	0.572	0.561	0.164
DS-31 (KT-FT)	0.529	0.413	0.337	0.282	0.532	0.363	0.020	0.398	0.290	0.223	0.175	0.516	0.507	0.241

Table 3. End-to-end fine-tuning quality on BeyondDialogue and Llemr.

System Resource	KT-FT	Routed	Shared
Attention Rank (GPU)	8	8	8
Shared Expert Rank (GPU)	8	8	205
Routed Experts Rank (CPU)	8	9	8
LoRA Expert Rank (GPU)	256	N/A	N/A
Total Trainable Parameters	4.1B	4.1B	4.1B
Hot Experts in GPU Memory	9	10	9

Table 4. Resource allocation for Standard LoRA on QW-35. KT-FT denotes the configuration with LE enabled.

reduction as LE. The Routed Control increases the rank of the existing routed adapters to match the total trainable parameters. The Shared Control increases the rank of the existing shared adaptation to match the active adaptation parameters per token. As an example, Table 4 lists the QW-35 resource allocation for Standard LoRA with LE and the two controls.

Time-to-Quality. Following DAWNBench [4, 5] and MLPerf Training [36], TTQ is defined as the wall-clock time required to reach a comparable quality target and reflects the convergence efficiency. For each LoRA method, we set the quality target to 5% above the fully converged validation loss.

Figure 11 shows that KT-FT with LE reduces TTQ by a factor of at least 1.9 relative to both the Routed Control and the Shared Control across different LoRA algorithms, showing that LE can be combined with these distinct update rules.

In the same fine-tuning runs, LE accounts for 19.3–20.6% of the measured MoE gradient norm across the four LoRA methods, whereas routed experts account for 99.0–100% in the two controls. The existing shared expert accounts for less than 1% even when its LoRA rank is increased in the Shared Control. Thus, despite a cold expert’s limited updates [56], a token routed to that expert also receives adaptation from LE, whose training includes tokens routed to other experts.

Model Quality. We evaluate final model quality on BeyondDialogue and Llemr using QW-35 and DS-31. We report precision with BLEU- n (B- n) [42], recall with ROUGE- n (R- n) [31], distinctness with Distinct- n (D- n) [25], and sequence coherence with ROUGE-L (R-L). As shown in Table 3, at full convergence, models fine-tuned with LE outperform the base models and the two controls on most metrics across both models and workloads.

6 Real-World Deployment Experiences

KT-FT has been adopted by enterprises adapting models for downstream applications, enterprise foundation model training teams, and laboratory researchers. This section reports experiences in local deployments with limited GPU resources.

Downstream Adaptation for Enterprises. Small and medium-sized enterprises often require local execution to maintain data privacy when adapting MoE models for proprietary tasks, such as document QA and vertical assistants. However, limited GPU resources constrain local deployments. In one deployment case, the technical team at Hisense Group fine-tuned a model using KT-FT on eight RTX 4090 GPUs. Compared with a baseline framework on 16 A100 GPUs, KT-FT reduced the average end-to-end completion time from 22.4 to 14.3 hours, providing a 1.6× speedup. It also reduced the direct compute cost per iteration by 81.5%. Moreover, across repeated adaptation tasks, KT-FT reuses expert weights, achieving efficient multi-LoRA fine-tuning and inference.

Foundation Model Research and Development. The Joy AI team used KT-FT for LoRA fine-tuning to explore and test new ideas. On RTX 4090 GPUs, the team fine-tuned its 700B-parameter MoE with 65K-token contexts, achieving a 5% gain in performance. KT-FT allowed the team to validate new algorithms on these consumer GPUs and iterate quickly. The team also used KT-FT to generate rollouts for agentic post-training and validate complex rollout workflows. KT-FT served as a backend for the team’s own frameworks, which were similar to verl [48].

Model Experiments for Laboratory. Several laboratory teams used KT-FT for local experiments with limited resources. In a study of adaptive memory in emotional conversation, one team fine-tuned Qwen3.5-397B on RTX 4090 using KT-FT. The study used 37,313 training samples and 4,145 test samples. Another team studied the generation of diverse advertisements from e-commerce product reviews, using a benchmark containing 88,608 reviews of 500 products. They used KT-FT to generate advertisements with models including Kimi-K2-Thinking, MiniMax-M2.5, and Qwen3-235B-A22B.

7 Related Work

Offloading and Expert Management. Offloading systems reduce GPU memory use by storing training state in CPU memory [10]. They improve training efficiency by coordinating data transfers [29] and overlapping transfers with computation [11, 62]. For distributed MoE training, expert placement [16, 39], prefetching [69], and token compression [38] reduce communication overhead. Buffer reuse [72] and activation recomputation [73] also reduce memory use. KT-FT reduces CPU-GPU weight transfers during the forward and backward passes by executing experts on the CPU and exchanging activations and their gradients.

CPU-GPU Co-computation. Inference systems reduce expert loading overhead through caching and prefetching [64] and use CPUs for model computation [2, 37, 77]. Adapter serving improves GPU utilization through request batching [3, 49] and balances load through request migration [58]. During prefill, CPU execution of adapters can also overlap adapter loading [27]. Training systems also use CPUs for parameter updates [23, 28]. KT-FT supports MoE fine-tuning through CPU-GPU co-computation, combining layout optimization, expert scheduling, and batched parameter updates.

Parameter-Efficient Fine-Tuning. Low-rank updates reduce the number of trainable parameters while keeping base model weights frozen [9, 17, 71]. Related methods route among adaptation experts [24, 30, 70] or reuse low-rank components across experts [68, 74]. The system-algorithm co-design in KT-FT, named LoRA Expert, couples shared adaptation with CPU-GPU co-computation, using residual GPU memory for a routing-independent branch that processes every token in parallel with routed CPU experts to reduce TTQ.

8 Conclusion

We presented KT-FT, a system for local LoRA fine-tuning of trillion-parameter MoE models on CPU-GPU platforms. Executing experts on CPUs avoids repeated PCIe weight transfers, while layout and scheduling optimizations reduce overheads from incompatible layouts and routing skew. KT-FT achieves $2.5\times$ – $21.2\times$ the throughput of offloading baselines with standard LoRA. KT-FT also accelerates full-parameter

fine-tuning of 100B-class MoE models. Users can adapt MoE models with limited GPU resources while keeping data on private infrastructure and reducing costs.

Acknowledgments

We thank all developers who contributed to fine-tuning support in the KTransformers community. We especially thank Hao Wang of Joy AI and Chenhui Xiong of Hisense Group for helping test KT-FT’s LoRA fine-tuning, full-parameter fine-tuning, and reinforcement learning functionality, and for providing the real-world deployment data reported in this paper.

References

- [1] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Maksim Ribinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. 2023. Petals: Collaborative Inference and Fine-tuning of Large Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Danushka Bollegala, Ruihong Huang, and Alan Ritter (Eds.). Association for Computational Linguistics, Toronto, Canada, 558–568. doi:10.18653/v1/2023.acl-demo.54
- [2] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. 2025. KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP ’25)*. Association for Computing Machinery, New York, NY, USA, 1014–1029. doi:10.1145/3731569.3764843
- [3] Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2023. Punica: Multi-Tenant LoRA Serving. doi:10.48550/arXiv.2310.18547 arXiv:2310.18547 [cs].
- [4] Cody Coleman, Daniel Kang, Deepak Narayanan, Luigi Nardi, Tian Zhao, Jian Zhang, Peter Bailis, Kunle Olukotun, Chris Ré, and Matei Zaharia. 2019. Analysis of DAWNBench, a Time-to-Accuracy Machine Learning Performance Benchmark. *SIGOPS Oper. Syst. Rev.* 53, 1 (July 2019), 14–25. doi:10.1145/3352020.3352024
- [5] Cody A. Coleman, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Christopher Ré, and Matei A. Zaharia. 2017. DAWNBench: An End-to-End Deep Learning Benchmark and Competition. In *NIPS 2017 Workshop on Machine Learning Systems*. <https://dawn.cs.stanford.edu/publications/dawnbench/dawnbench-end-end-deep-learning-benchmark-and-competition>
- [6] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. arXiv:2401.06066 [cs.CL] <https://arxiv.org/abs/2401.06066>
- [7] DeepSeek-AI et al. 2024. DeepSeek-V3 Technical Report. doi:10.48550/arXiv.2412.19437 DeepSeek-V3 technical report, arXiv:2412.19437 [cs].
- [8] DeepSeek-AI et al. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. doi:10.48550/arXiv.2606.19348 DeepSeek-V4 technical report, arXiv:2606.19348 [cs].
- [9] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314 [cs.LG] <https://arxiv.org/abs/2305.14314>
- [10] Jiarui Fang, Zilin Zhu, Shenggui Li, Hui Su, Yang Yu, Jie Zhou, and Yang You. 2023. Parallel Training of Pre-Trained Models via Chunk-Based Dynamic Memory Management. *IEEE Transactions on Parallel*

- and Distributed Systems 34, 1 (Jan. 2023), 304–315. doi:10.1109/TPDS.2022.3219819 arXiv:2108.05818 [cs].
- [11] Yangyang Feng, Minhui Xie, Zijie Tian, Shuo Wang, Youyou Lu, and Jiwu Shu. 2023. Mobius: Fine Tuning Large-Scale Models on Commodity GPU Servers. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. Association for Computing Machinery, New York, NY, USA, 489–501. doi:10.1145/3575693.3575703
 - [12] Aryo Gema, Pasquale Minervini, Luke Daines, Tom Hope, and Beatrice Alex. 2024. Parameter-Efficient Fine-Tuning of LLaMA for the Clinical Domain. In *Proceedings of the 6th Clinical Natural Language Processing Workshop*, Tristan Naumann, Asma Ben Abacha, Steven Bethard, Kirk Roberts, and Danielle Bitterman (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 91–104. doi:10.18653/v1/2024.clinicalnlp-1.9
 - [13] GLM-5-Team et al. 2026. GLM-5: from Vibe Coding to Agentic Engineering. doi:10.48550/arXiv.2602.15763 GLM-5 technical report, arXiv:2602.15763 [cs].
 - [14] Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. 2022. Accelerate: Training and inference at scale made simple, efficient and adaptable. <https://github.com/huggingface/accelerate>.
 - [15] Soufiane Hayou, Nikhil Ghosh, and Bin Yu. 2026. PLoP: Precise LoRA Placement for Efficient Finetuning of Large Models. In *International Conference on Learning Representations*, C. Vondrick, B. Har-iharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (Eds.), Vol. 2026. 65166–65191. https://proceedings.iclr.cc/paper_files/paper/2026/file/6ace8b8873adf5af379e8b5e0dcd8664-Paper-Conference.pdf
 - [16] Chenghao Hu, Yufei Kang, and Baochun Li. 2025. Communication-Efficient MoE Fine-Tuning with Locality-Aware Expert Placement. In *2025 IEEE 45th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 166–176. doi:10.1109/ICDCS63083.2025.00025
 - [17] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. doi:10.48550/arXiv.2106.09685 arXiv:2106.09685 [cs].
 - [18] Hanbo Huang, Yihan Li, Bowen Jiang, Bo Jiang, Lin Liu, Ruoyu Sun, Zhuotao Liu, and Shiyu Liang. 2025. A Middle Path for On-Premises LLM Deployment: Preserving Privacy Without Sacrificing Model Confidentiality. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 8321–8359. doi:10.18653/v1/2025.emnlp-main.420
 - [19] Shijia Jiang, Yongfu Dai, Haochen Jia, Yuxin Wang, and Hao Wang. 2025. IntelliChain Stars at the Regulations Challenge Task: A Large Language Model for Financial Regulation. In *Proceedings of the Joint Workshop of the 9th Financial Technology and Natural Language Processing (FinNLP), the 6th Financial Narrative Processing (FNP), and the 1st Workshop on Large Language Models for Finance and Legal (LLMFinLegal)*, Chung-Chi Chen, Antonio Moreno-Sandoval, Jimin Huang, Qianqian Xie, Sophia Ananiadou, and Hsin-Hsi Chen (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 371–384. <https://aclanthology.org/2025.finnlp-1.43/>
 - [20] Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. 2025. Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models. In *International Conference on Learning Representations*, Vol. 2025. 56099–56115. https://proceedings.iclr.cc/paper_files/paper/2025/file/8cd1ce03ea58b3d7dfd809e4d42f08ea-Paper-Conference.pdf
 - [21] Yechan Kim, Hwijoon Lim, and Dongsu Han. 2024. Scaling Beyond the GPU Memory Limit for Large Mixture-of-Experts Model Training. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. 24342–24353. <https://proceedings.mlr.press/v235/kim24w.html>
 - [22] Kimi Team et al. 2026. Kimi K3: Open Frontier Intelligence. doi:10.48550/arXiv.2607.24653 Kimi K3 technical report, arXiv:2607.24653 [cs].
 - [23] Tingfeng Lan, Yusen Wu, Bin Ma, Zhaoyuan Su, Rui Yang, Tekin Bicer, Masahiro Tanaka, Olatunji Ruwase, Dong Li, and Yue Cheng. 2025. ZenFlow: Enabling Stall-Free Offloading Training via Asynchronous Updates. doi:10.48550/arXiv.2505.12242 arXiv:2505.12242 [cs] version: 3.
 - [24] Dengchun Li, Yingzi Ma, Naizheng Wang, Zhengmao Ye, Zhiyuan Cheng, Yinghao Tang, Yan Zhang, Lei Duan, Jie Zuo, Cal Yang, and Mingjie Tang. 2024. MixLoRA: Enhancing Large Language Models Fine-Tuning with LoRA-based Mixture of Experts. doi:10.48550/arXiv.2404.15159 arXiv:2404.15159 [cs].
 - [25] Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. 2016. A Diversity-Promoting Objective Function for Neural Conversation Models. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Kevin Knight, Ani Nenkova, and Owen Rambow (Eds.). Association for Computational Linguistics, San Diego, California, 110–119. doi:10.18653/v1/N16-1014
 - [26] Rumeng Li, Xun Wang, and Hong Yu. 2024. LlamaCare: An Instruction Fine-Tuned Large Language Model for Clinical NLP. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue (Eds.). ELRA and ICCL, Torino, Italia, 10632–10641. <https://aclanthology.org/2024.lrec-main.930/>
 - [27] Suyi Li, Hanfeng Lu, Tianyuan Wu, Minchen Yu, Qizhen Weng, Xusheng Chen, Yizhou Shan, Binhang Yuan, and Wei Wang. 2025. Toppings: CPU-Assisted, Rank-Aware Adapter Serving for LLM Inference. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, 613–629. <https://www.usenix.org/conference/atc25/presentation/li-suyi-toppings>
 - [28] Xinyu Lian, Masahiro Tanaka, Olatunji Ruwase, and Minjia Zhang. 2025. SuperOffload: Unleashing the Power of Large-Scale LLM Training on Superchips. doi:10.48550/arXiv.2509.21271 arXiv:2509.21271 [cs].
 - [29] Changyue Liao, Mo Sun, Zihan Yang, Jun Xie, Kaiqi Chen, Binhang Yuan, Fei Wu, and Zeke Wang. 2025. Ratel: Optimizing Holistic Data Movement to Fine-tune 100B Model on a Consumer GPU. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 292–306. doi:10.1109/ICDE65448.2025.00029 ISSN: 2375-026X.
 - [30] Mengqi Liao, Wei Chen, Junfeng Shen, Shengnan Guo, and Huaiyu Wan. 2025. HMoRA: Making LLMs More Effective with Hierarchical Mixture of LoRA Experts. In *International Conference on Learning Representations*, Yisong Yue, Animesh Garg, Nanyun Peng, Fei Sha, and Rose Yu (Eds.), Vol. 2025. 91309–91330. https://proceedings.iclr.cc/paper_files/paper/2025/file/e43a33994a28f746dcfd53eb51ed3c2d-Paper-Conference.pdf
 - [31] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*. Association for Computational Linguistics, Barcelona, Spain, 74–81. <https://aclanthology.org/W04-1013/>
 - [32] Yuhang Liu, Tao Li, Zhehao Huang, Zuopeng Yang, and Xiaolin Huang. 2026. Bi-LoRA: Efficient Sharpness-Aware Minimization for Fine-Tuning Large-Scale Models. In *International Conference on Learning Representations*, Vol. 2026. 96782–96813. <https://openreview.net/forum?id=zoYPlgX1bH>

- [33] Yilun Liu, Yunpu Ma, Yuetian Lu, Shuo Chen, Zifeng Ding, and Volker Tresp. 2026. Parameter-Efficient Routed Fine-Tuning: Mixture-of-Experts Demands Mixture of Adaptation Modules. In *Findings of the Association for Computational Linguistics: EACL 2026*, Vera Demberg, Kentaro Inui, and Lluís Marquez (Eds.). Association for Computational Linguistics, Rabat, Morocco, 4439–4457. doi:10.18653/v1/2026.findings-eacl.232
- [34] YongKang Liu, Yiqun Zhang, Qian Li, Tong Liu, Shi Feng, Daling Wang, Yifei Zhang, and Hinrich Schuetze. 2024. HiFT: A Hierarchical Full Parameter Fine-Tuning Strategy. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 18266–18287. doi:10.18653/v1/2024.emnlp-main.1015
- [35] Kai Lv, Yuqing Yang, Tengxiao Liu, Qipeng Guo, and Xipeng Qiu. 2024. Full Parameter Fine-tuning for Large Language Models with Limited Resources. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 8187–8198. doi:10.18653/v1/2024.acl-long.445
- [36] Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Mikičevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, David Brooks, Dehao Chen, Debojyoti Dutta, Udit Gupta, Kim Hazelwood, Andrew Hock, Xinyuan Huang, Atsushi Ike, Bill Jia, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Guokai Ma, Deepak Narayanan, Tayo Oguntebi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St. John, Tsuguchika Tabaru, Carole-Jean Wu, Lingjie Xu, Masafumi Yamazaki, Cliff Young, and Matei Zaharia. 2020. MLPerf Training Benchmark. arXiv:1910.01500 [cs.LG] <https://arxiv.org/abs/1910.01500>
- [37] Seonjin Na, Geonhwa Jeong, Byung Hoon Ahn, Aaron Jezghani, Jeffrey Young, Christopher J Hughes, Tushar Krishna, and Hye-soon Kim. 2025. FlexInfer: Flexible LLM Inference with CPU Computations. In *Proceedings of Machine Learning and Systems*, Vol. 7. MLSys. https://proceedings.mlsys.org/paper_files/paper/2025/file/698cfaf72a208aef2e78bcac55b74328-Paper-Conference.pdf
- [38] Xiaonan Nie, Qibin Liu, Fangcheng Fu, Shenhan Zhu, Xupeng Miao, Xiaoyang Li, Yang Zhang, Shouda Liu, and Bin Cui. 2024. LSH-MoE: Communication-efficient MoE Training via Locality-Sensitive Hashing. doi:10.48550/arXiv.2411.08446 arXiv:2411.08446 [cs].
- [39] Xiaonan Nie, Xupeng Miao, Zilong Wang, Zichao Yang, Jilong Xue, Lingxiao Ma, Gang Cao, and Bin Cui. 2023. FlexMoE: Scaling Large-scale Sparse Pre-trained Model Training via Dynamic Device Placement. *Proceedings of the ACM on Management of Data* 1, 1 (May 2023), 1–19. doi:10.1145/3588964 arXiv:2304.03946 [cs].
- [40] Isabella Olariu, Cedric Lothritz, Jacques Klein, Tegawendé Bissyandé, Siwen Guo, and Shohreh Haddadan. 2023. Evaluating Parameter-Efficient Finetuning Approaches for Pre-trained Models on the Financial Domain. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 15482–15491. doi:10.18653/v1/2023.findings-emnlp.1035
- [41] oneDNN Contributors. [n. d.]. oneAPI Deep Neural Network Library (oneDNN). <https://github.com/uxlfoundation/oneDNN> Accessed October 4, 2026.
- [42] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, Pierre Isabelle, Eugene Charniak, and Dekang Lin (Eds.). Association for Computational Linguistics, Philadelphia, Pennsylvania, USA, 311–318. doi:10.3115/1073083.1073135
- [43] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc., 12 pages.
- [44] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. <https://qwen.ai/blog?id=qwen3.5>
- [45] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. doi:10.48550/arXiv.2104.07857 arXiv:2104.07857 [cs].
- [46] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. doi:10.48550/arXiv.2101.06840 arXiv:2101.06840 [cs].
- [47] John Schulman and Thinking Machines Lab. 2025. LoRA Without Regret. *Thinking Machines Lab: Connectionism* (Sept. 2025). doi:10.64434/tml.20250929 <https://thinkingmachines.ai/blog/lora/>.
- [48] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. HybridFlow: A Flexible and Efficient RLHF Framework. In *Proceedings of the Twentieth European Conference on Computer Systems*. ACM, 1279–1297. doi:10.1145/3689031.3696075
- [49] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. 2024. SLoRA: Scalable Serving of Thousands of LoRA Adapters. In *Proceedings of Machine Learning and Systems*, Vol. 6. 296–311. https://proceedings.mlsys.org/paper_files/paper/2024/file/906419cd502575b617cc489a1a696a67-Paper-Conference.pdf
- [50] Ilias Stogiannidis, Stavros Vassos, Prodromos Malakasiotis, and Ion Androutopoulos. 2023. Cache me if you Can: an Online Cost-aware Teacher-Student framework to Reduce the Calls to Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 14999–15008. doi:10.18653/v1/2023.findings-emnlp.1000
- [51] Xiaoyang Sun, Wei Wang, Shenghao Qiu, Renyu Yang, Songfang Huang, Jie Xu, and Zheng Wang. 2022. STRONGHOLD: Fast and Affordable Billion-Scale Deep Learning Model Training. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–17. doi:10.1109/sc41404.2022.00076
- [52] Hanyin Wang, Chufan Gao, Bolun Liu, Qiping Xu, Guleid Hussein, Mohamad El Labban, Kingsley Iheasirim, Hariprasad Reddy Korsapati, Chuck Outcalt, and Jimeng Sun. 2025. Towards Adapting Open-Source Large Language Models for Expert-Level Clinical Note Generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 12084–12117. doi:10.18653/v1/2025.findings-acl.626
- [53] Jianwei Wang, Chengming Shi, Junyao Yang, Haoran Li, Qianli Ma, Huiping Zhuang, Cen Chen, and Ziqian Zeng. 2025. RewardDS: Privacy-Preserving Fine-Tuning for Large Language Models via Reward Driven Data Synthesis. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 4479–4500. doi:10.18653/v1/2025.emnlp-main.223
- [54] Xintao Wang, Heng Wang, Yifei Zhang, Xinfeng Yuan, Rui Xu, Jen-Tse Huang, Siyu Yuan, Haoran Guo, Jiangjie Chen, Shuchang Zhou, Wei Wang, and Yanghua Xiao. 2025. CoSER: Coordinating LLM-Based Persona Simulation of Established Roles. In *Proceedings*

- of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267), Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (Eds.). PMLR, 64822–64858. <https://proceedings.mlr.press/v267/wang25dk.html>
- [55] Zihan Wang, Deli Chen, Damai Dai, Runxin Xu, Zhuoshu Li, and Yu Wu. 2024. Let the Expert Stick to His Last: Expert-Specialized Fine-Tuning for Sparse Architectural Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 784–801. doi:10.18653/v1/2024.emnlp-main.46
- [56] Jia Wei, Zhonghao Zhang, Ping Chen, Qianyang Li, Yancheng Pan, Shaoxun Wang, Ziyi Qiu, and Longxiang Wang. 2026. HELLoRA: Hot Experts Layer-Level Low-Rank Adaptation for Mixture-of-Experts Models. doi:10.48550/ARXIV.2605.18795
- [57] Yuanxin Wei, Jiangsu Du, Jiazhi Jiang, Xiao Shi, Xianwei Zhang, Dan Huang, Nong Xiao, and Yutong Lu. 2024. APTMoE: Affinity-Aware Pipeline Tuning for MoE Models on Bandwidth-Constrained GPU Nodes. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14. doi:10.1109/SC41406.2024.00096
- [58] Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. dLoRA: Dynamically Orchestrating Requests and Adapters for LoRA LLM Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 911–927. <https://www.usenix.org/conference/osdi24/presentation/wu-bingyang>
- [59] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhjanj Kambadur, David Rosenberg, and Gideon Mann. 2023. BloombergGPT: A Large Language Model for Finance. doi:10.48550/arXiv.2303.17564 arXiv:2303.17564 [cs].
- [60] Zhenbang Wu, Anant Dadu, Mike Nalls, Faraz Faghri, and Jimeng Sun. 2024. Instruction Tuning Large Language Models to Understand Electronic Health Records. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 54772–54786. doi:10.52202/079017-1737
- [61] An Yang et al. 2025. Qwen3 Technical Report. doi:10.48550/arXiv.2505.09388 Qwen3 technical report, arXiv:2505.09388 [cs].
- [62] Ruijia Yang and Zeyi Wen. 2026. An Efficient Heterogeneous Co-Design for Fine-Tuning on a Single GPU. doi:10.48550/arXiv.2603.16428 arXiv:2603.16428 [cs].
- [63] Dianhai Yu, Liang Shen, Hongxiang Hao, Weibao Gong, Huachao Wu, Jiang Bian, Lirong Dai, and Haoyi Xiong. 2024. MoESys: A Distributed and Efficient Mixture-of-Experts Training and Inference System for Internet Services. *IEEE Transactions on Services Computing* 17, 5 (Sept. 2024), 2626–2639. doi:10.1109/TSC.2024.3399654
- [64] Hanfei Yu, Xingqi Cui, Hong Zhang, Hao Wang, and Hao Wang. 2025. Taming Latency-Memory Trade-Off in MoE-Based LLM Serving via Fine-Grained Expert Offloading. doi:10.48550/arXiv.2502.05370 arXiv:2502.05370 [cs].
- [65] Xin Yu, Yujia Wang, Jinghui Chen, and Lingzhou Xue. 2025. Alt-LoRA: Towards Better Gradient Approximation in Low-Rank Adaptation with Alternating Projections. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., 37484–37522. doi:10.52202/085713-1258
- [66] Yeyong Yu, Runsheng Yu, Haojie Wei, Zhanqiu Zhang, and Quan Qian. 2025. Beyond Dialogue: A Profile-Dialogue Alignment Framework Towards General Role-Playing Language Model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 11992–12022. doi:10.18653/v1/2025.acl-long.586
- [67] Zhengqing Yuan, Hanchi Sun, Lichao Sun, and Yanfang Ye. 2026. MegaTrain: Full Precision Training of 100B+ Parameter Large Language Models on a Single GPU. arXiv preprint arXiv:2604.05091. arXiv:2604.05091 [cs.CL] doi:10.48550/arXiv.2604.05091
- [68] Hanqing Zeng, Yinglong Xia, Zhuokai Zhao, Chuan Jiang, Qiang Zhang, Jiayi Liu, Qunshu Zhang, Lizhu Zhang, Xiangjun Fan, and Benyu Zhang. 2025. S'MoRE: Structural Mixture of Residual Experts for Parameter-Efficient LLM Fine-tuning. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., 81441–81478. doi:10.52202/085713-2721
- [69] Junyi Zhang, Chuanhu Ma, Xiong Wang, Yuntao Nie, Yuqing Li, Yuedong Xu, Xiaofei Liao, Bo Li, and Hai Jin. 2025. PopFetcher: Towards Accelerated Mixture-of-Experts Training Via Popularity Based Expert-Wise Prefetch. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, 1053–1069. <https://www.usenix.org/conference/atc25/presentation/zhang-junyi>
- [70] Jingfan Zhang, Yi Zhao, Dan Chen, Xing Tian, Huanran Zheng, and Wei Zhu. 2024. MiLoRA: Efficient Mixture of Low-Rank Adaptation for Large Language Models Fine-tuning. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, Miami, Florida, USA, 17071–17084. doi:10.18653/v1/2024.findings-emnlp.994
- [71] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023. Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=lq62uWRJjiY>
- [72] Zheng Zhang, Donglin Yang, Yaqi Xia, Liang Ding, Dacheng Tao, Xiaobo Zhou, and Dazhao Cheng. 2023. MPipeMoE: Memory Efficient MoE for Pre-trained Models with Adaptive Pipeline Parallelism. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 167–177. doi:10.1109/IPDPS54959.2023.00026 arXiv:2506.22175 [cs].
- [73] Lu Zhao, Rong Shi, Shaoqing Zhang, Yueqiang Chen, Baoguo He, Hongfeng Sun, Ziqing Yin, Shangchao Su, Zhiyan Cui, Liang Dong, Xiyuan Li, Lingbin Wang, Jianwei He, Jiesong Ma, Weikang Huang, Jianglei Tong, Dongdong Gao, Jian Zhang, Hong Tian, Hui Shen, Zongtai Luo, Zhaoqun Sun, Hongxing Niu, and Yue Sun. 2025. MemFine: Memory-Aware Fine-Grained Scheduling for MoE Training. doi:10.48550/arXiv.2511.21431 arXiv:2511.21431 [cs]; revised January 2026.
- [74] Lulu Zhao, Weihao Zeng, Xiaofeng Shi, and Hua Zhou. 2025. MoSLD: An Extremely Parameter-Efficient Mixture-of-Shared LoRAs for Multi-Task Learning. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 1647–1659. <https://aclanthology.org/2025.coling-main.111/>
- [75] Yanli Zhao, Andrew Gu, Rohan Varna, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. doi:10.48550/arXiv.2304.11277 arXiv:2304.11277 [cs].
- [76] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. 2024. LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models. doi:10.48550/arXiv.2403.13372 arXiv:2403.13372 [cs].
- [77] Zeyu Zhu, Gang Li, Peisong Wang, Zitao Mo, Minnan Pei, Zhuoran Song, Xiaoyao Liang, and Jian Cheng. 2026. DALI: A Workload-Aware Offloading Framework for Efficient MoE Inference on Local PCs. doi:10.48550/arXiv.2602.03495 arXiv:2602.03495 [cs].